

Sprint 11 — Proxy Pattern

Class 1

Learning Objectives

By the end of this class, we should be able to:

- Explain **why Proxy Pattern exists** and what problem it solves.
- Identify the roles of **Subject, Real Subject, and Proxy**.
- Understand how a Proxy can **control access** to another object.
- Implement a basic Proxy using a shared interface.
- Explain and implement common Proxy types:
 - Protection Proxy
 - Virtual Proxy
 - Remote Proxy
 - Caching Proxy
- Understand how Proxy can **stop, delay, redirect, or reuse access** to the real object.
- Compare **Proxy vs Decorator, Adapter, Facade, and Chain of Responsibility**.
- Explain how Proxy supports **SRP, OCP, and DIP**.
- Recognize appropriate enterprise use cases such as authorization, caching, remote-service access, lazy loading, and monitoring.
- Identify common Proxy mistakes and over-engineering.
- Design and implement a practical **secured/caching file-storage Proxy** as homework.

The main idea we should remember is:

Proxy provides a substitute or representative for another object and controls access to it.

Think of it as:

```
Client
|
Proxy
|
Real Object
```

The client often does not know whether it is talking to the real object or the proxy.

Lesson 1 — Why Proxy Exists

Suppose we have a document service:

```
public interface DocumentService {

    String readDocument(
        String documentId
    );
}
```

Real implementation:

```
public class RealDocumentService
    implements DocumentService {

    @Override
    public String readDocument(
        String documentId
    ) {
        System.out.println(
```

```

        "Loading document from storage..."
    );

    return "Sensitive document content";
}
}

```

Client:

```

DocumentService service = new RealDocumentService();

String content = service.readDocument("DOC-001");

```

At first, everything looks fine.

But then business says:

Only authorized users can read the document.

We could modify RealDocumentService.

```

public class RealDocumentService
    implements DocumentService {

    @Override
    public String readDocument(
        String documentId
    ) {

        checkPermission();

        return loadDocument(
            documentId
        );
    }
}

```

Now security logic is inside document-loading logic.

Then more requirements arrive:

- Authorization
- Audit logging
- Caching
- Lazy loading
- Remote access
- Access statistics
- Rate limiting

Our real service keeps growing.

Lesson 2 — The Problem With Direct Access

Without Proxy:

```
Client
|
RealDocumentService
```

The client accesses the real object directly.

If we need access control, we often start polluting the real service:

```
public String readDocument(
    String documentId
) {

    authenticate();

    authorize();

    checkRateLimit();
```

```
    audit();

    return loadDocument(
        documentId
    );
}
```

Now RealDocumentService has too many responsibilities.

Lesson 3 — Proxy Idea

Instead:

```
Client
|
DocumentServiceProxy
|
RealDocumentService
```

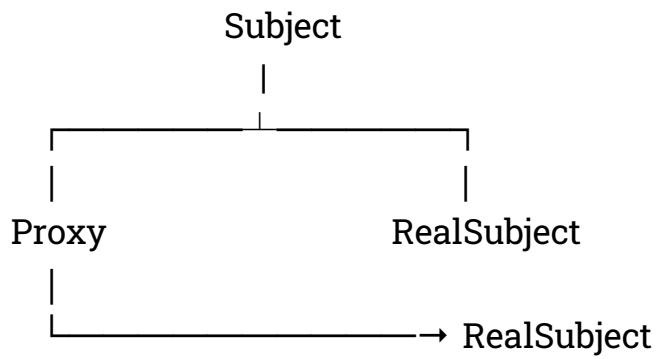
The proxy implements the same interface:

```
public class DocumentServiceProxy
    implements DocumentService {
}
```

So the client still depends on: DocumentService
not the concrete implementation.

Lesson 4 — Proxy Structure

Classic structure:



In Java:

```
public interface DocumentService {  
  
    String readDocument(  
        String documentId  
    );  
}
```

Real subject:

```
public class RealDocumentService  
    implements DocumentService {  
  
    @Override  
    public String readDocument(  
        String documentId  
    ) {  
        return "Document content";  
    }  
}
```

Proxy:

```
public class DocumentServiceProxy  
    implements DocumentService {  
  
    private final DocumentService target;
```

```

public DocumentServiceProxy(
    DocumentService target
) {
    this.target = target;
}

@Override
public String readDocument(
    String documentId
) {

    return target.readDocument(
        documentId
    );
}
}

```

At this point the proxy does nothing special yet.

But now we have a place to control access.

Lesson 5 — Protection Proxy

A very common Proxy type is **Protection Proxy**.

Its job:

Control whether the client is allowed to access the real object.

Example:

```

public class SecuredDocumentServiceProxy
    implements DocumentService {

    private final DocumentService target;
    private final CurrentUser currentUser;

```

```

public SecuredDocumentServiceProxy(
    DocumentService target,
    CurrentUser currentUser
) {
    this.target = target;
    this.currentUser = currentUser;
}

@Override
public String readDocument(
    String documentId
) {
    if (!currentUser
        .permissions()
        .contains(
            "DOCUMENT_READ"
        )) {
        throw new SecurityException(
            "DOCUMENT_READ permission required"
        );
    }

    return target.readDocument(
        documentId
    );
}
}

```

Now:

Client

|

SecuredDocumentServiceProxy

|

Permission check

|

RealDocumentService

Lesson 6 — Client Does Not Change

Client still uses:

DocumentService service;

It may receive:

```
new RealDocumentService()
```

or:

```
new SecuredDocumentServiceProxy(  
    new RealDocumentService(),  
    currentUser  
)
```

The client doesn't need to know.

That is a major benefit.

Lesson 7 — Full Example

Subject:

```
public interface PaymentService {  
  
    PaymentResult pay(  
        PaymentRequest request  
    );  
}
```

Real service:

```
public class RealPaymentService  
    implements PaymentService {  
  
    @Override  
    public PaymentResult pay(  
        PaymentRequest request  
    ) {  
  
        System.out.println(  
            "Processing payment: "  
            + request.amount()  
        );  
  
        return new PaymentResult(  
            true,  
            "PAYMENT-001"  
        );  
    }  
}
```

Proxy:

```
public class PaymentSecurityProxy  
    implements PaymentService {  
  
    private final PaymentService target;
```

```

private final CurrentUser currentUser;

public PaymentSecurityProxy(
    PaymentService target,
    CurrentUser currentUser
) {
    this.target = target;
    this.currentUser = currentUser;
}

@Override
public PaymentResult pay(
    PaymentRequest request
) {
    if (!currentUser.permissions()
        .contains(
            "PAYMENT_CREATE"
        )) {
        throw new SecurityException(
            "PAYMENT_CREATE permission required"
        );
    }

    return target.pay(
        request
    );
}
}

```

Lesson 8 — Proxy Can Run Logic Before the Target

```

@Override
public PaymentResult pay(
    PaymentRequest request
) {

    authorize();

    return target.pay(
        request
    );
}

```

Flow:

```

Client
|
Proxy
|
Authorize
|
RealPaymentService

```

Lesson 9 — Proxy Can Run Logic After the Target

```

@Override
public PaymentResult pay(
    PaymentRequest request
) {

    PaymentResult result =
        target.pay(request);

    audit(result);
}

```

```
    return result;
}
```

Flow:

```
Client
|
Proxy
|
Real Service
|
Audit
|
Client
```

Lesson 10 — Proxy Can Avoid Calling the Real Object

This is very important.

Suppose the user is unauthorized.

```
if (!authorized()) {
    throw new SecurityException();
}
```

Then:

```
Proxy
|
Authorization (X)
|
STOP
```

Real object is never called.

This is different from a Decorator, where delegation is usually part of adding behavior.

Lesson 11 — Caching Proxy

Another useful type is **Caching Proxy**.

```
public class CachingDocumentServiceProxy
    implements DocumentService {

    private final DocumentService target;

    private final Map<String, String> cache =
        new HashMap<>();

    public CachingDocumentServiceProxy(
        DocumentService target
    ) {
        this.target = target;
    }

    @Override
    public String readDocument(
        String documentId
    ) {

        if (cache.containsKey(
            documentId
        )) {

            System.out.println(
                "Return document from cache"
            );
```

```

        return cache.get(
            documentId
        );
    }

    String content =
        target.readDocument(
            documentId
        );

    cache.put(
        documentId,
        content
    );

    return content;
}

```

First call:

Client -> Proxy -> Cache MISS -> Real Service

Second call:

Client -> Proxy -> Cache HIT -> Return directly

The real object is not called.

Lesson 12 — Virtual Proxy

Virtual Proxy delays creation of an expensive object.

Suppose loading a large image is expensive.

Bad:

```
Image image =  
    new HighResolutionImage(  
        "hotel.jpg"  
    );
```

The expensive object is created immediately.

Instead:

```
public interface Image {  
  
    void display();  
}
```

Real object:

```
public class HighResolutionImage  
    implements Image {  
  
    private final String filename;  
  
    public HighResolutionImage(  
        String filename  
    ) {  
        this.filename = filename;  
  
        loadFromDisk();  
    }  
  
    private void loadFromDisk() {  
        System.out.println(  
            "Loading large image: "  
                + filename  
        );  
    }  
}
```

```

@Override
public void display() {
    System.out.println(
        "Display image: "
        + filename
    );
}
}

```

Proxy:

```

public class ImageProxy
    implements Image {

    private final String filename;

    private HighResolutionImage realImage;

    public ImageProxy(
        String filename
    ) {
        this.filename = filename;
    }

    @Override
    public void display() {

        if (realImage == null) {

            realImage =
                new HighResolutionImage(
                    filename
                );
        }

        realImage.display();
    }
}

```

```
}  
}
```

Now object creation is delayed.

Lesson 13 — Virtual Proxy Flow

Creation:

```
Image image =  
    new ImageProxy(  
        "hotel.jpg"  
    );
```

At this moment:

Large image NOT loaded yet.

Only when:

```
image.display();
```

we get:

```
Proxy  
|  
Create Real Image  
|  
Load Image  
|  
Display
```

Second call:

```
image.display();
```

uses the already-loaded real image.

Lesson 14 — Remote Proxy

This is extremely important for enterprise systems.

Imagine:

Booking Service -> Payment Service

But Payment Service is another microservice.

The client would like to think in terms of:

```
public interface PaymentClient {  
  
    PaymentResponse pay(  
        PaymentRequest request  
    );  
}
```

Implementation may actually make an HTTP request:

```
public class HttpPaymentClient  
    implements PaymentClient {  
  
    @Override  
    public PaymentResponse pay(  
        PaymentRequest request  
    ) {  
  
        // HTTP request to remote service  
  
        return sendHttpRequest(  

```

```

        request
    );
}
}

```

Conceptually:

```

BookingService
|
PaymentClient Proxy
|
HTTP
|
Payment Microservice

```

The local object represents a remote object.

That is the idea of a **Remote Proxy**.

Lesson 15 — Real Enterprise Example

Suppose:

```

public interface UserClient {

    UserResponse getUser(
        UUID userId
    );
}

```

Application code:

```

UserResponse user =
    userClient.getUser(
        userId
    );

```

);

But implementation might:

serialize request

|

send HTTP

|

wait for network

|

deserialize response

The caller doesn't care.

It interacts with:

URLConnection

as though it were a local object.

Lesson 16 — Proxy Types

We should know these important types.

Protection Proxy

Controls permissions.

Client -> Security Proxy -> Real Service

Example:

Authorization

Role checks

Permission checks

Virtual Proxy

Delays expensive object creation.

Client -> Lazy Proxy -> Create Real Object When Needed

Examples:

Large image

Large document

Expensive resource

Remote Proxy

Represents an object in another process/server.

Client -> Remote Proxy -> Network -> Remote Service

Examples:

REST client

RPC

gRPC stub

Feign-style client

Caching Proxy

Avoids repeated access to expensive resources.

Client

|

Cache Proxy

—— HIT -> return

—— MISS -> Real Service

Logging / Monitoring Proxy

Controls access while recording calls.

Client -> Monitoring Proxy -> Real Service

Lesson 17 — Proxy vs Direct Service Modification

Bad approach:

```
public class PaymentService {  
  
    public PaymentResult pay(  
        PaymentRequest request  
    ) {  
  
        authorize();  
  
        log();  
  
        checkCache();  
    }  
}
```

```
        rateLimit();

        process();

        audit();

        return result;
    }
}
```

Now one class handles:

- Business processing
- Security
- Logging
- Caching
- Rate limiting
- Auditing

This violates SRP.

Proxy separates those concerns.

Lesson 18 — Proxy and SRP

Real service:

```
public class RealPaymentService {

    public PaymentResult pay(
        PaymentRequest request
    ) {

        // payment business logic
    }
}
```

```
}
```

Security proxy:

```
public class SecurityProxy {  
  
    // security responsibility  
}
```

Caching proxy:

```
public class CachingProxy {  
  
    // caching responsibility  
}
```

Each class has a clearer reason to change.

Lesson 19 — Proxy and OCP

Suppose we already have:

PaymentService

Then business asks:

Add access control.

We can add:

SecurityPaymentProxy

without changing:

RealPaymentService

Another requirement:

Add caching.

Add:

CachingPaymentProxy

Again, no need to change the real service.

Lesson 20 — Proxy and DIP

Client depends on:

PaymentService

not:

RealPaymentService

So:

```
public class PaymentController {  
  
    private final PaymentService paymentService;  
  
    public PaymentController(  
        PaymentService paymentService  
    ) {  
        this.paymentService =  
            paymentService;  
    }  
}
```

The actual object might be:

RealPaymentService

or:

SecurityProxy

or:

CachingProxy

The controller doesn't care.

Lesson 21 — Proxy vs Decorator

This comparison is essential because the structure is very similar.

Both may look like:

```
public class SomeWrapper
    implements PaymentService {

    private final PaymentService target;
}
```

So how do we tell them apart?

By **intent**.

Decorator Intent

Add behavior

Example:

PaymentGateway -> RetryDecorator -> MetricsDecorator

The purpose is enriching behavior.

Proxy Intent

Control access

Example:

Client -> SecurityProxy -> RealPaymentService

Proxy decides whether, when, or how access happens.

Decorator says: "Let me add behavior."

Proxy says: "You must go through me to reach the real object."

Lesson 22 — Proxy May Look Exactly Like Decorator

Example:

```
public class LoggingPaymentProxy
    implements PaymentService {

    private final PaymentService target;
}
```

Could this also be called a Decorator?

Possibly.

This is why we should not identify patterns only from class diagrams.

Ask:

What is the design intent?

If it primarily adds optional behavior:

Decorator

If it primarily represents or controls access to another object:

Proxy

Intent matters more than syntax.

Lesson 23 — Proxy vs Adapter

Adapter:

Our Interface -> Adapter -> Incompatible API

Purpose: Change interface.

Example:

PaymentGateway -> AbaPaymentAdapter -> ABA SDK

Proxy:

Same Interface -> Proxy -> Real Object

Purpose: Control access.

Easy rule:

Adapter changes interface. Proxy preserves interface.

Lesson 24 — Proxy vs Facade

Facade:

Client

|

BookingFacade

—— RoomService

—— PaymentService

—— NotificationService

Facade simplifies a subsystem.

Proxy:

Client -> PaymentProxy -> PaymentService

Proxy represents one underlying object/service and controls access to it.

Easy rule:

Facade simplifies many objects. Proxy stands in front of one object.

Lesson 25 — Proxy vs Chain of Responsibility

Chain:

Request

|

Validation

|

Authentication

|

Authorization

|

Business

Request flows through several handlers.

Proxy:

Client -> Proxy ->Target

One object represents another.

The proxy may internally perform several checks, but its design role is different.

Lesson 26 — When Proxy Is a Good Choice

Use Proxy when you need:

- authorization
- lazy initialization
- remote object access
- caching
- access logging
- rate limiting
- monitoring
- expensive resource control
- transparent access control

Lesson 27 — When Not to Use Proxy

Don't introduce Proxy simply because:

“Design Patterns are good.”

If:

```
service.doSomething();
```

needs no access control, remote representation, lazy loading, or lifecycle management, then direct access may be better.

Unnecessary Proxy can make code harder to trace.

Lesson 28 — Common Mistake: Putting Business Logic in Proxy

Bad:

```
public class PaymentProxy {  
  
    public PaymentResult pay(  
        PaymentRequest request  
    ) {  
  
        calculateTax();  
  
        calculateDiscount();  
  
        createInvoice();  
  
        updateReservation();  
    }  
}
```

```
        return realService.pay(  
            request  
        );  
    }  
}
```

Now the proxy is becoming the application service.

Proxy should primarily handle access-related concerns.

Business workflow belongs elsewhere.

Lesson 29 — Common Mistake: Proxy Interface Does Not Match Target

Classic Proxy should usually expose the same abstraction.

Good:

```
public interface DocumentService {  
}
```

Both:

RealDocumentService

and:

DocumentServiceProxy

implement it.

If the wrapper changes the interface, you may actually be implementing Adapter.

Lesson 30 — Common Mistake: Client Creates Real Object Directly

Bad:

```
RealPaymentService realService =  
    new RealPaymentService();
```

```
PaymentService proxy =  
    new PaymentSecurityProxy(  
        realService  
    );
```

throughout application code.

Then clients still know too much.

Better:

```
PaymentService paymentService;
```

and let configuration decide which implementation is injected.

This becomes especially important in Spring.

Lesson 31 — Common Mistake: Too Many Nested Proxies

Imagine:

```
SecurityProxy  
|  
LoggingProxy  
|  
CachingProxy  
|  
RateLimitProxy  
|
```

```
MetricsProxy
|
AuditProxy
|
RealService
```

This can become hard to understand.

At some point cross-cutting concerns may be better handled with:

- framework middleware
- interceptors
- Spring AOP
- filters

Lesson 32 — Enterprise Example: Property Document Download

Imagine Room Cambodia has owner verification documents.

```
public interface VerificationDocumentService {

    byte[] download(
        UUID documentId
    );
}
```

Real service:

```
public class S3VerificationDocumentService
    implements VerificationDocumentService {

    @Override
    public byte[] download(
        UUID documentId
    ) {
```

```

        return loadFromS3(
            documentId
        );
    }
}

```

But these documents are private.

We don't want arbitrary callers accessing them.

Proxy:

```

public class SecuredVerificationDocumentProxy
    implements VerificationDocumentService {

    private final VerificationDocumentService target;

    private final PermissionChecker permissionChecker;

    public SecuredVerificationDocumentProxy(
        VerificationDocumentService target,
        PermissionChecker permissionChecker
    ) {
        this.target = target;
        this.permissionChecker =
            permissionChecker;
    }

    @Override
    public byte[] download(
        UUID documentId
    ) {

        permissionChecker.require(
            "OWNER_VERIFICATION_DOCUMENT_READ"
        );
    }
}

```

```
        return target.download(  
            documentId  
        );  
    }  
}
```

Now:

```
Controller  
|  
VerificationDocumentService  
|  
Security Proxy  
|  
S3 Service
```

This is an excellent enterprise example.

Lesson 33 — Proxy Can Combine With Other Patterns

Proxy and Adapter:

```
Client  
|  
Security Proxy  
|  
Payment Adapter  
|  
ABA SDK
```

Proxy controls access.

Adapter integrates an incompatible API.

Proxy and Decorator:

Client

|

Security Proxy

|

Metrics Decorator

|

Retry Decorator

|

Payment Adapter

|

Remote SDK

Again, each pattern has a different purpose.

Homework — Protected File Storage

Build:

```
public interface FileStorage {  
  
    byte[] download(  
        String fileId  
    );  
  
    void upload(  
        String fileId,  
        byte[] data  
    );  
}
```

Real implementation:

LocalFileStorage

Then implement:

SecuredFileStorageProxy

Rules:

FILE_READ

FILE_WRITE

For download:

FILE_READ required

For upload:

FILE_WRITE required

Homework Part 2 — Caching Proxy

Add:

CachingFileStorageProxy

Behavior:

First download:

Cache MISS -> Real Storage

Second download:

Cache HIT -> Return directly

Homework Part 3 — Virtual Proxy

Create:

```
public interface Report {  
  
    void display();  
}
```

Real object:

LargeFinancialReport

Pretend creation takes several seconds.

Create:

FinancialReportProxy

The real report should only be created on the first:

display();

Class 1 Interview Questions

1. What problem does Proxy solve?
2. What are Subject, Real Subject, and Proxy?
3. Why does Proxy usually implement the same interface as the real object?
4. What is a Protection Proxy?
5. What is a Virtual Proxy?
6. What is a Remote Proxy?

7. What is a Caching Proxy?
8. Can a Proxy decide not to call the real object?
9. Proxy vs Decorator?
10. Proxy vs Adapter?
11. Proxy vs Facade?
12. Proxy vs Chain of Responsibility?
13. How does Proxy support SRP?
14. How does Proxy support OCP?
15. What are the disadvantages of Proxy?
16. Give enterprise examples of Proxy.

Class 1 Summary

Without Proxy:

Client

|

Real Service

The client has direct access.

With Proxy:

Client

|

Proxy

|

Real Service

The Proxy decides:

Can I call it?

Should I call it?

When should I create it?

Should I return cached data instead?

Should this become a remote request?

Proxy stands in place of the real object and controls how the client reaches it.